

2º curso / 2º cuatr.
Grados en
Ing. Informática

Arquitectura de Computadores

Tema 1

Arquitecturas Paralelas: Clasificación y Prestaciones

Material elaborado por Mancia Anguita

Profesores: Mancia Anguita, Maribel García y Christian Morillas



**UNIVERSIDAD
DE GRANADA**



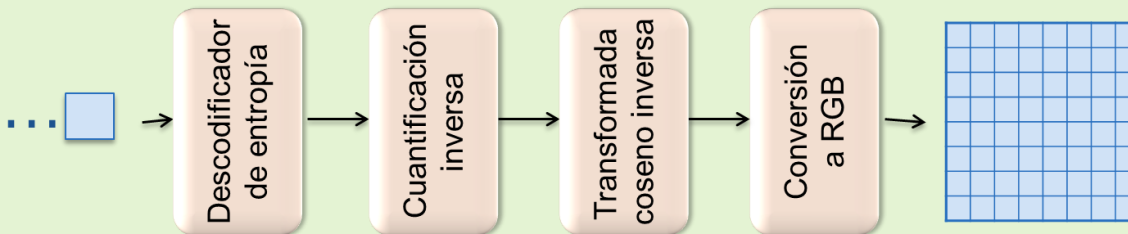
ICAR

INGENIERÍA DE COMPUTADORES,
AUTOMÁTICA Y ROBÓTICA

Lecciones

- Lección 1. Clasificación del paralelismo implícito en una aplicación
- Lección 2. Clasificación de arquitecturas paralelas
- Lección 3. Evaluación de prestaciones

Estructura de funciones de un decodificador JPEG



Suma de vectores
 $c = a + b$

Producto de matrices
 $C = A * B$

```
Func1 () { ... }
Func2 () { ...
    for (i=0; i<N; i=++) {
        código para i
    }
}
Func3 () { ... }
Main () {
    ...
    Func1 (...);
    Func2 (...);
    Func3 (...);
    ...
}
```


Objetivos Lección 1

- Conocer las clasificaciones usuales del paralelismo implícito en una aplicación. Distinguir entre paralelismo de tareas y paralelismo de datos.
- Distinguir entre dependencias RAW, WAW, WAR.
- Distinguir entre hilo o hebra (*thread*) y proceso.
- Relacionar el paralelismo implícito en una aplicación con el nivel en el que se hace explícito para que se pueda utilizar (instrucción, thread, proceso) y con las arquitecturas paralelas que lo aprovechan.

Bibliografía

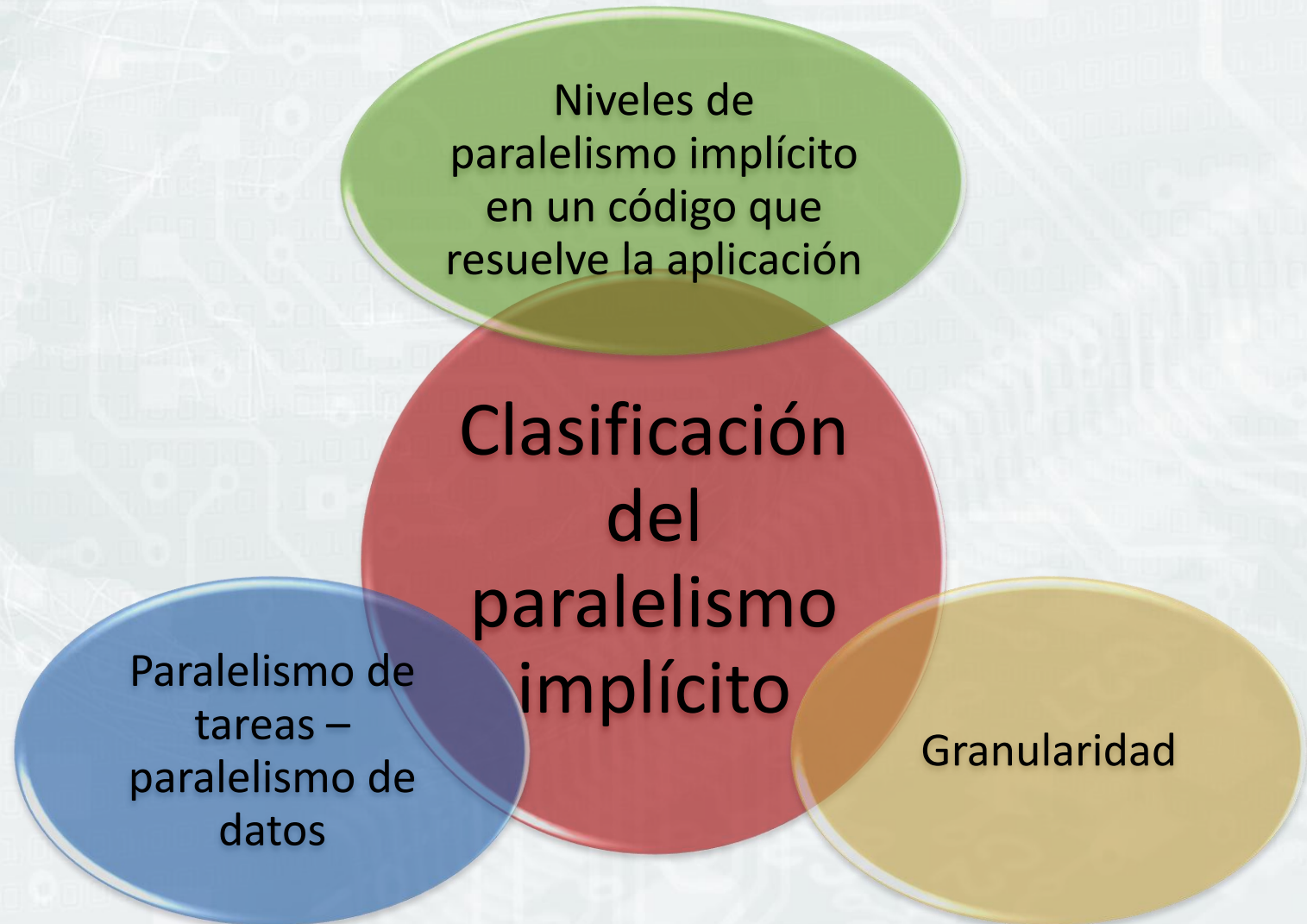
➤ Fundamental

- Sección 7.1. J. Ortega, M. Anguita, A. Prieto. *Arquitectura de Computadores*, Thomson, 2005. ESIIT/C.1 ORT arq

➤ Complementaria

- Secciones 3.7.1, 3.7.2. T. Rauber, G. Ränder. *Parallel Programming: for Multicore and Cluster Systems*. Springer 2023. Disponible en línea (biblioteca UGR):
<https://link.springer.com/book/10.1007/978-3-031-28924-8>

Criterios de clasificaciones del paralelismo implícito en una aplicación



Niveles de paralelismo implícito en una aplicación

Niveles:

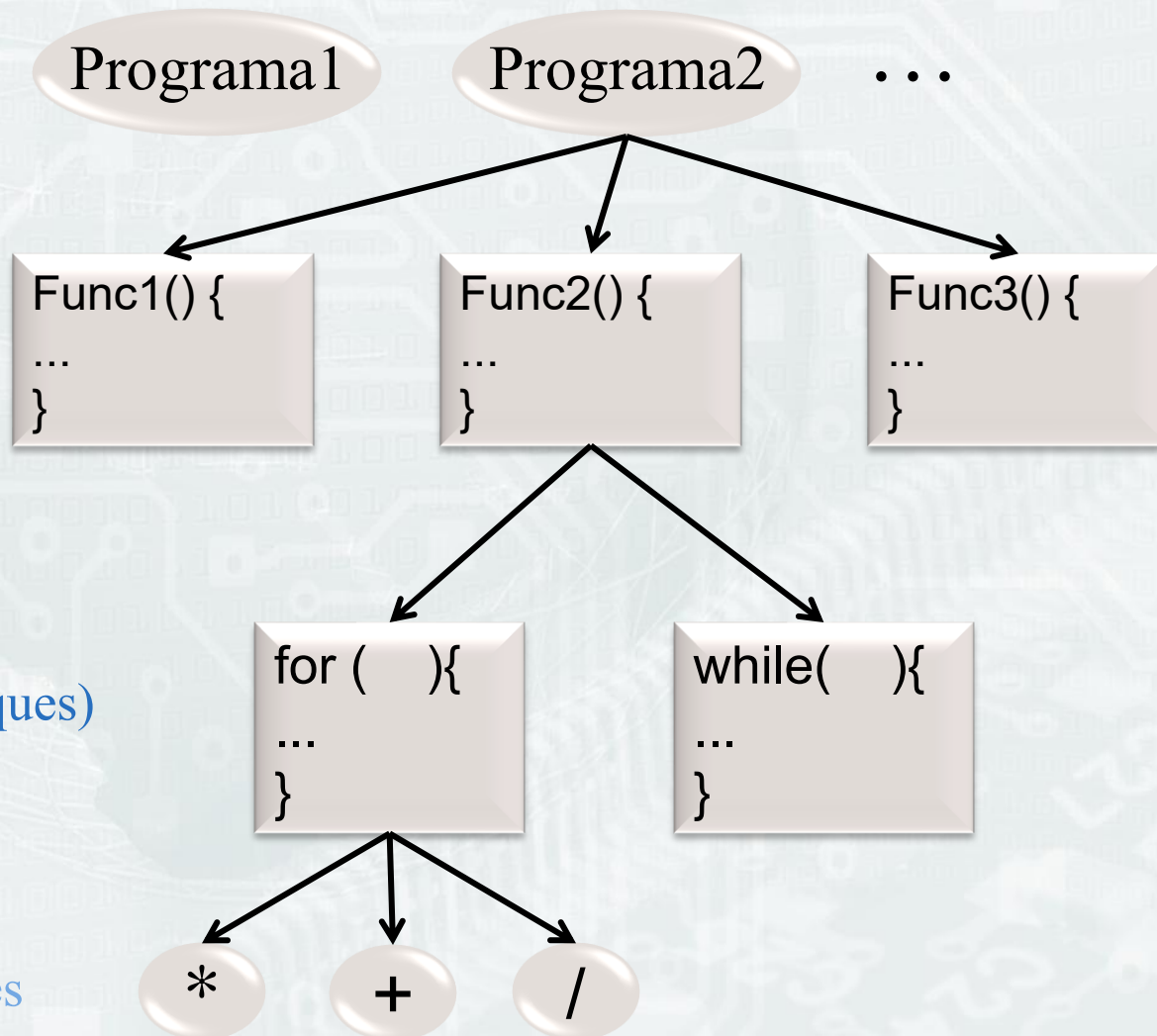
Programas



Funciones

Bucle (bloques)

Operaciones



Granularidad:

Grano Grueso

Grano Medio

Grano Medio-Fino

Grano Fino

Dependencias de datos

- Condiciones que se deben cumplir para que el bloque de código B_2 presente dependencia de datos con respecto a B_1 :
 - Deben hacer referencia a una misma posición de memoria M (variable).
 - B_1 aparece en la secuencia de código antes que B_2
- Tipos de dependencias de datos (de B_2 respecto a B_1):
 - RAW (*Read After Write*) o dependencia verdadera o de flujo
 - WAW (*Write After Write*) o **dependencia de salida** (dependencia de nombre)
 - WAR (*Write After Read*) o **anti-dependencia** (dependencia de nombre)



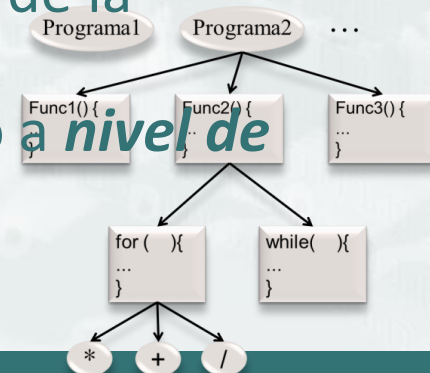
```
...  
a=b*c  
... //código que no modifica a  
d=a+c  
...
```

```
...  
a=b*c  
... //se lee a  
a=d+e  
... //se lee a
```

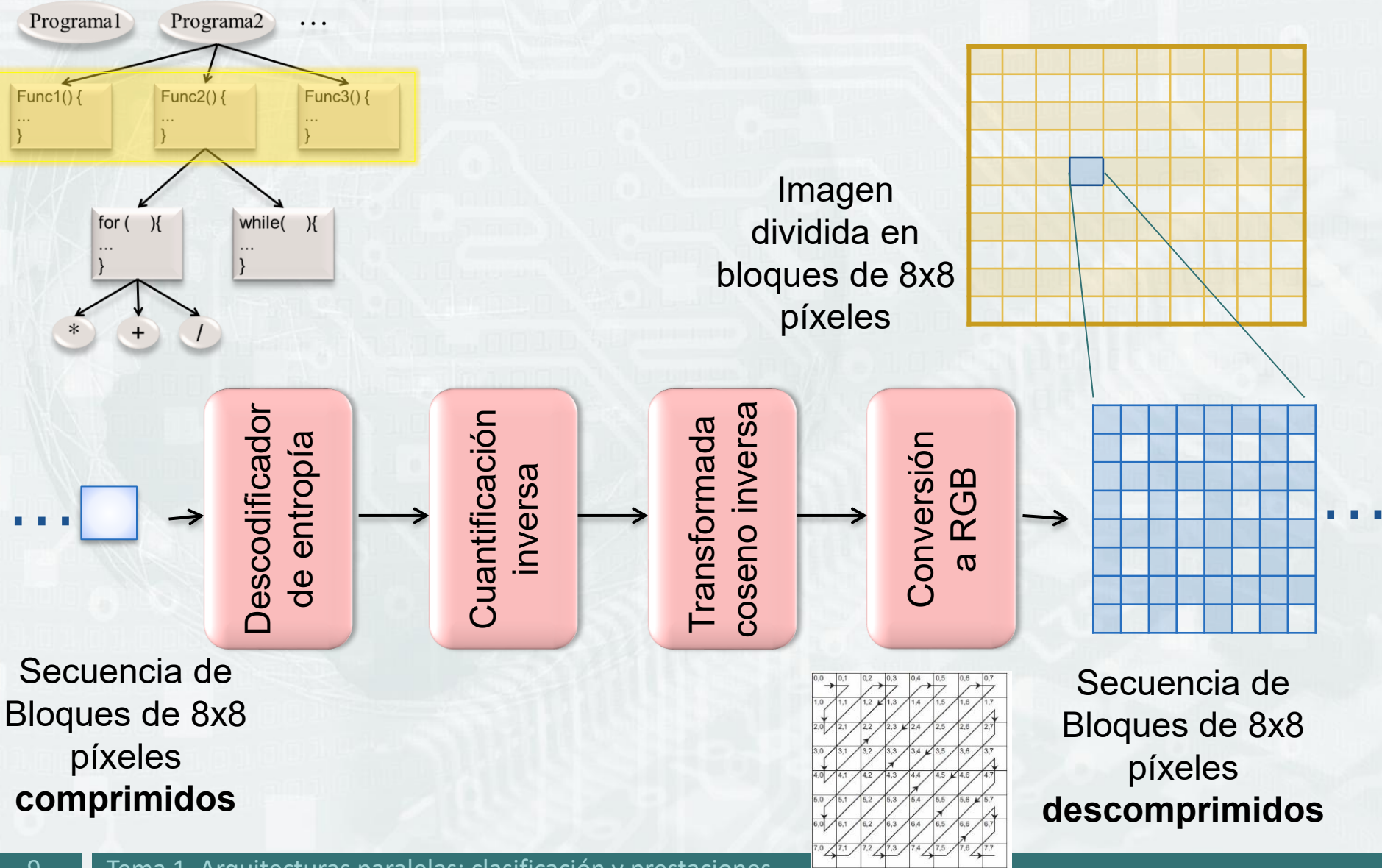
```
...  
b=a+1  
...  
a=d+e  
... //se lee a
```

Paralelismo implícito en una aplicación

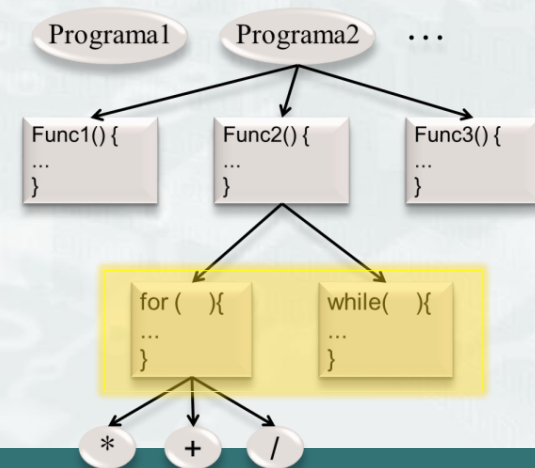
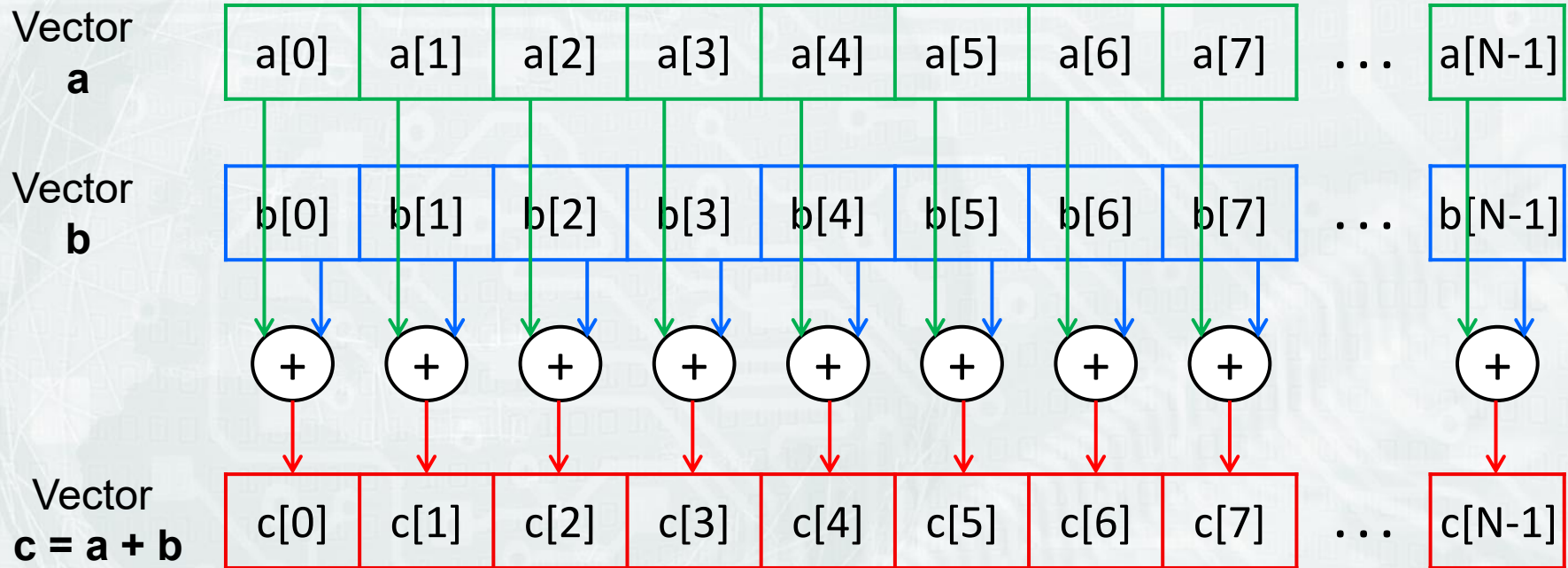
- Paralelismo de tareas (*task parallelism* o *TLP -Task Level Par.*)
 - Se encuentra extrayendo la estructura lógica de funciones de una aplicación.
 - Está relacionado con el paralelismo a **nivel de función**
- Paralelismo de datos (*data parallelism* o *DLP-Data Level Par.*)
 - Se encuentra implícito en las operaciones con estructuras de datos (vectores y matrices)
 - Por ejemplo, la operación vectorial $V1=V2+V3$ engloba múltiples sumas de escalares.
 - Se puede extraer de la representación matemática de la aplicación.
 - Está relacionado principalmente con el paralelismo a **nivel de bucle**



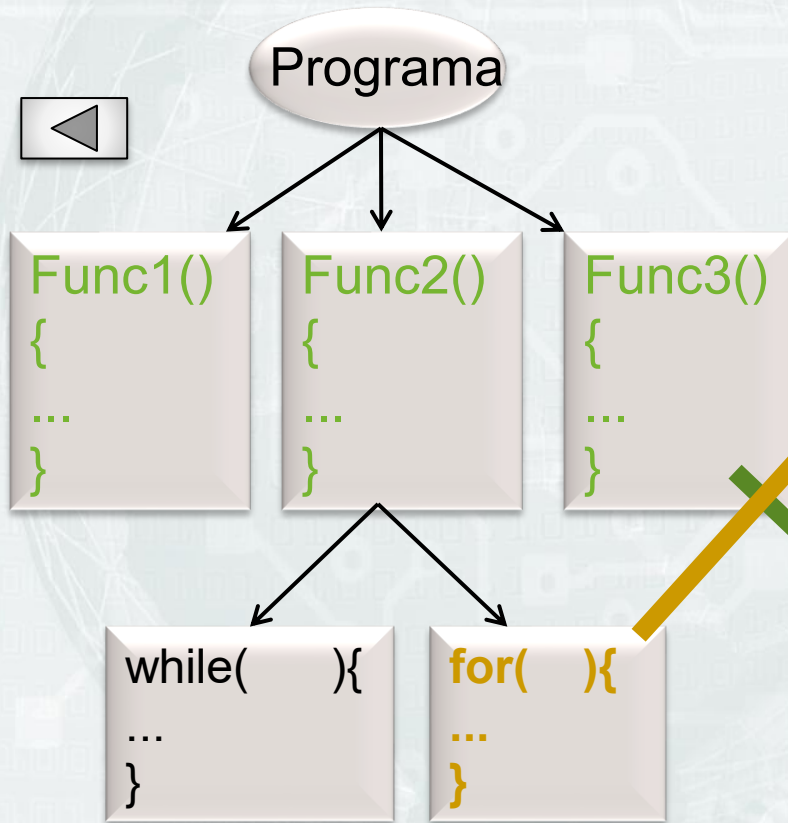
Estructura de funciones lógicas de una aplicación. Ej.: decodificador JPEG



Paralelismo de datos. Ej: suma de dos vectores

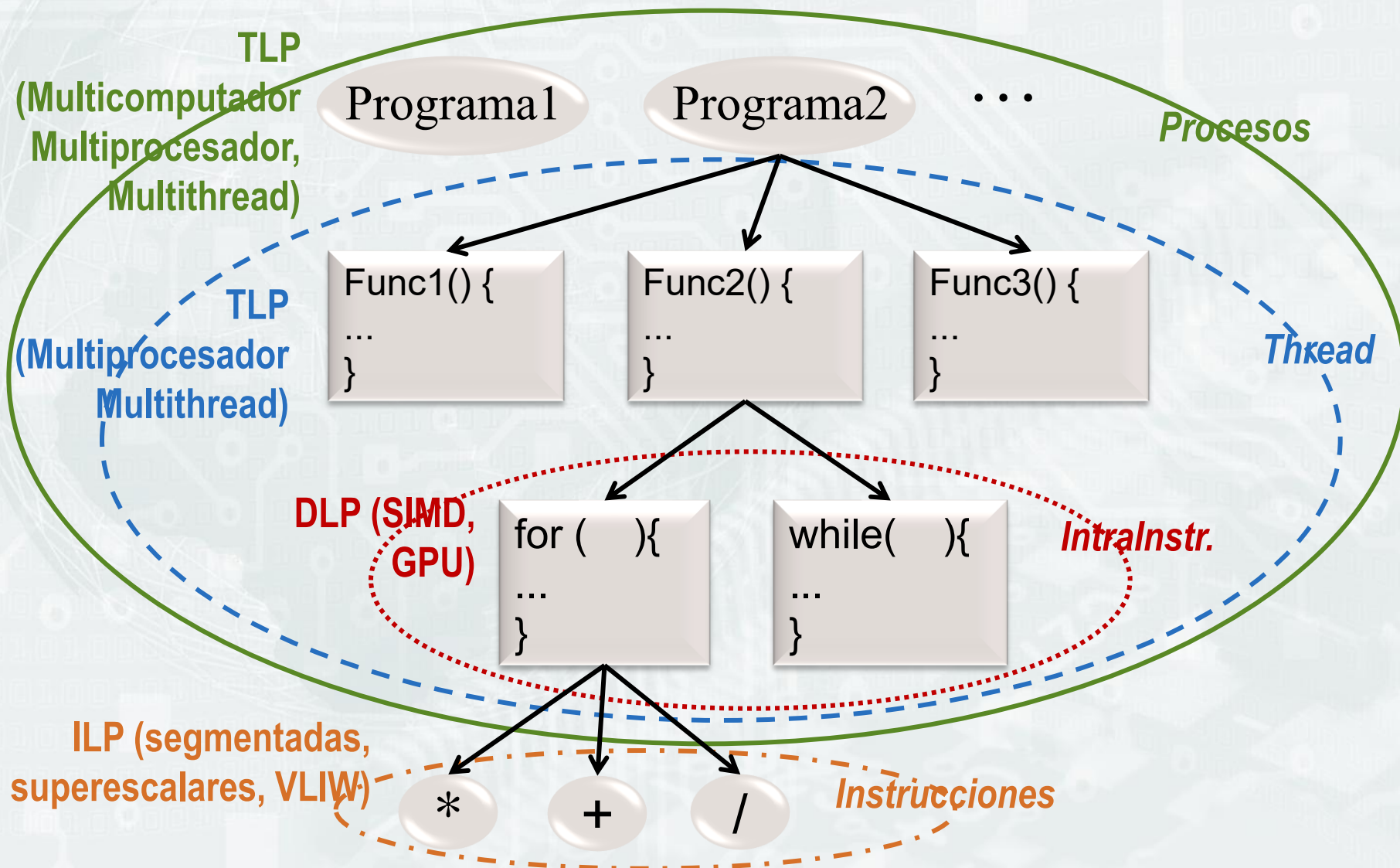


Paralelismo de datos y paralelismo de tareas en OpenMP (Prácticas 0, 1, 2 y 3)



```
Func1 () { ... }
Func2 () { ...
    #pragma omp parallel for
    for (i=0;i<N;i++){
        código para i
    }
    ... }
Func3 () { ... }
Main () {
    ...
    #pragma omp parallel sections
    { #pragma omp section
        Func1();
        #pragma omp section
        Func2();
        #pragma omp section
        Func3();
    }
    ... }
```


Paralelismo implícito (nivel de detección), explícito y arquitecturas paralelas



Nivel de paralelismo explícito.

Unidades en ejecución en un computador

➤ Instrucciones

- La unidad de control de un core o procesador gestiona la ejecución de instrucciones por la unidad de procesamiento

➤ *Thread* o *light process* (concepto del SO)

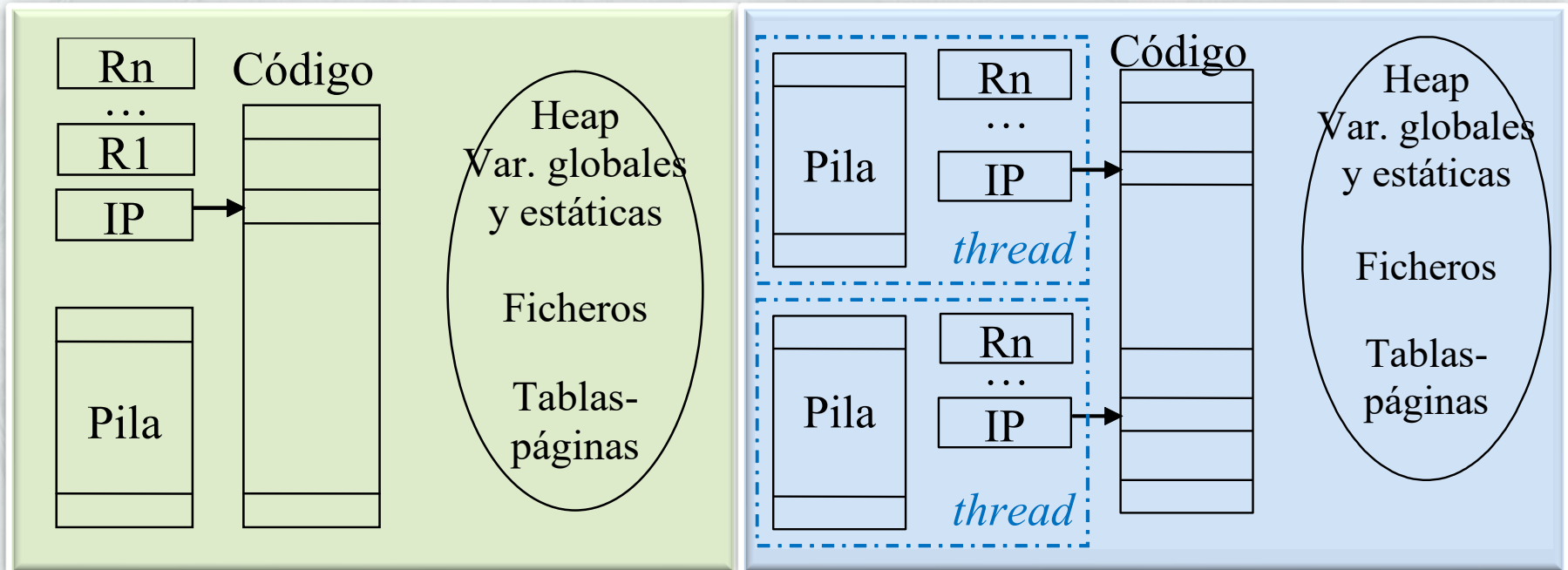
- Es la menor unidad de ejecución que gestiona el SO
- Menor secuencia de instrucciones que se pueden ejecutar en paralelo o concurrentemente

➤ Proceso o *process* (concepto del SO)

- Mayor unidad de ejecución que gestiona el SO
- Un proceso consta de uno o varios thread

Nivel de paralelismo explícito.

Threads versus procesos I



- **Proceso:** comprende el código del programa y todo lo que hace falta para su ejecución:
 - Datos en pila, segmentos (variables globales y estáticas) y en heap
 - Contenido de los registros
 - Tabla de páginas
 - Tabla de ficheros abiertos
- Para comunicar procesos hay que usar llamadas al SO
- Un proceso puede constar de múltiples flujos de instrucciones, llamados **threads** o procesos ligeros. Cada thread tiene:
 - Su propia pila
 - Contenido de los registros, en particular el contador de programa o *instruction pointer* y el puntero de pila o *stack pointer*
- Para comunicar threads de un proceso se usa la memoria que comparten

grano

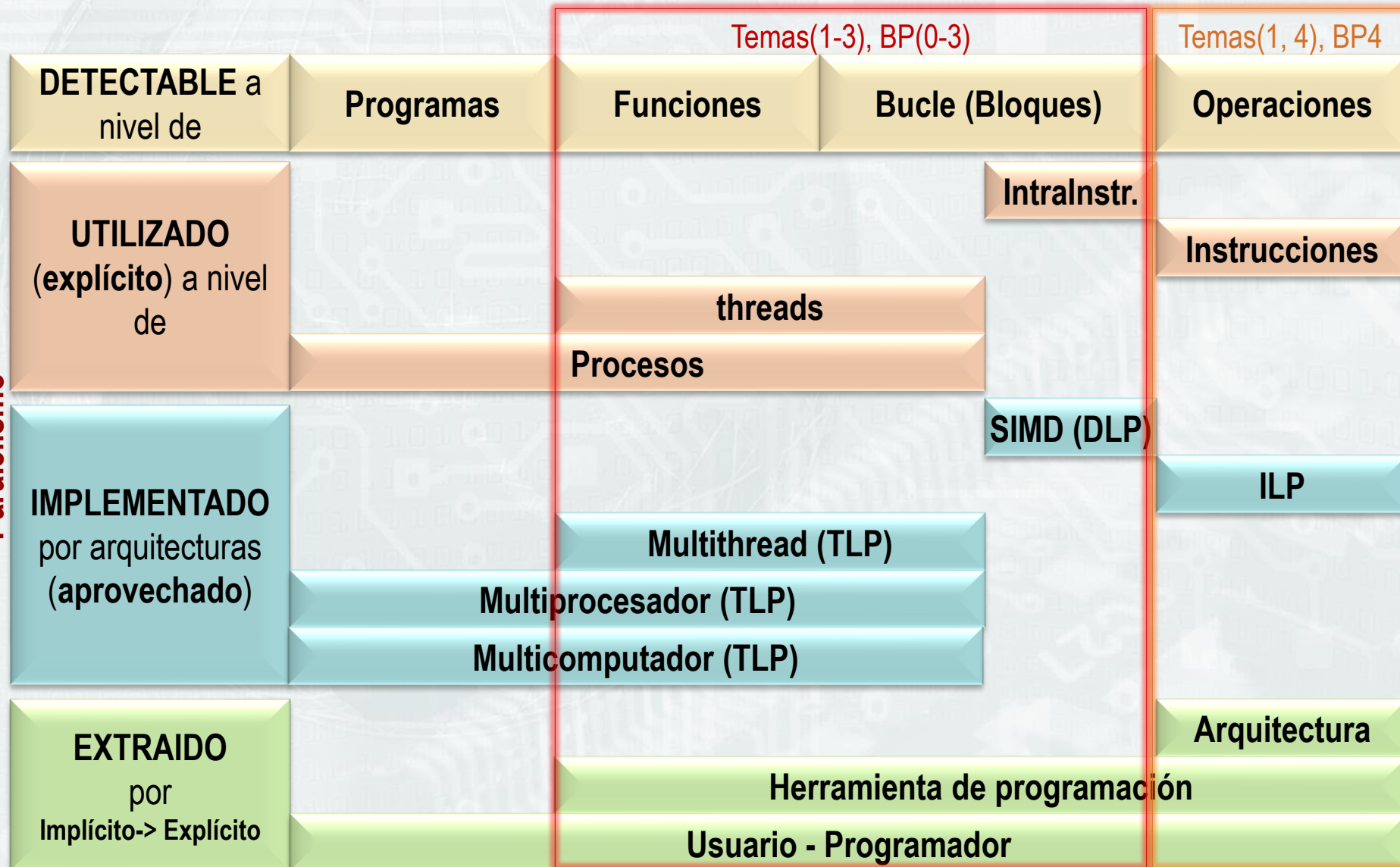
Nivel de paralelismo explícito. Threads versus procesos II



Detección, utilización, implementación y extracción del paralelismo

niveles

Paralelismo



Para ampliar

➤ Páginas Web:

- http://en.wikipedia.org/wiki/Instruction-level_parallelism
- https://en.wikipedia.org/wiki/Data_dependency
- http://en.wikipedia.org/wiki/Task_parallelism
- http://en.wikipedia.org/wiki/Data_parallelism

2º curso / 2º cuatr.
Grados en
Ing. Informática

Arquitectura de Computadores

Tema 1

Arquitecturas Paralelas: Clasificación y Prestaciones

Material elaborado por Mancia Anguita

Profesores: Mancia Anguita, Maribel García y Christian Morillas



**UNIVERSIDAD
DE GRANADA**



ICAR

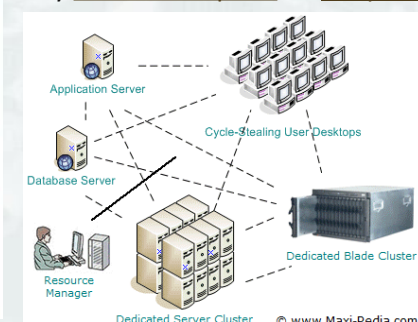
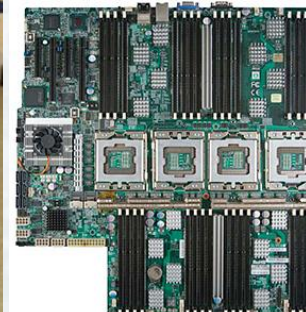
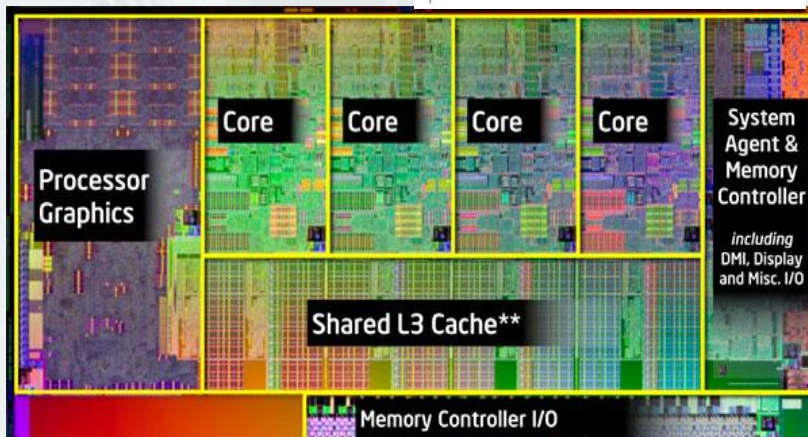
INGENIERÍA DE COMPUTADORES,
AUTOMÁTICA Y ROBÓTICA

Lecciones

- Lección 1. Clasificación del paralelismo implícito en una aplicación
- **Lección 2. Clasificación de arquitecturas paralelas**
- Lección 3. Evaluación de prestaciones



Photo by [unavailable parts](#) on [Unsplash](#)



Objetivos Lección 2

- Distinguir entre procesamiento o computación paralela y distribuida.
- Clasificar los computadores según segmento del mercado.
- Distinguir entre las diferentes clases de arquitecturas de la clasificación de Flynn.
- Diferenciar un multiprocesador de un multicomputador.
- Distinguir entre NUMA y SMP.
- Distinguir entre arquitecturas DLP, ILP y TLP.
- Distinguir entre arquitecturas TLP con una instancia de SO y TLP con varias instancias de SO.

Bibliografía

➤ Fundamental

- Secciones 1.3, 7.3 y 7.2.2. J. Ortega, M. Anguita, A. Prieto. *Arquitectura de Computadores*, Thomson, 2005. ESIIT/C.1
ORT arq

➤ Complementaria

- T. Rauber, G. Ränder. *Parallel Programming: for Multicore and Cluster Systems*. Springer 2010. Disponible en línea (biblioteca UGR): <http://dx.doi.org/10.1007/978-3-642-04818-0>
- Sección 1.2, pág. 28 John L. Hennessy, David A. Patterson, Christos Kozyrakis, *Computer Architecture : a Quantitative Approach*, Seventh edition., Waltham : Elsevier 2026.

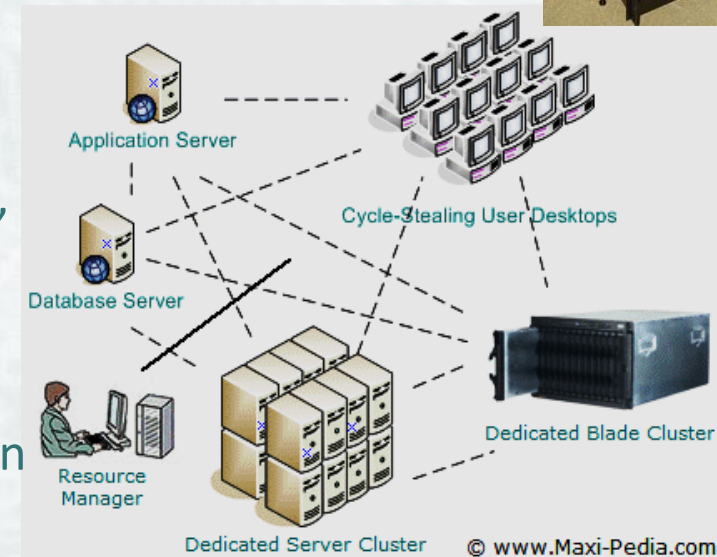
Contenidos

- Lección 1. Clasificación del paralelismo implícito en una aplicación
- **Lección 2. Clasificación de arquitecturas paralelas**
 - **Computación paralela y computación distribuida**
 - Clasificaciones de arquitecturas y sistemas paralelos
 - Nota histórica
- Lección 3. Evaluación de prestaciones

Computación paralela – Computación distribuida

Distributed Computing Environment from FOLDOC

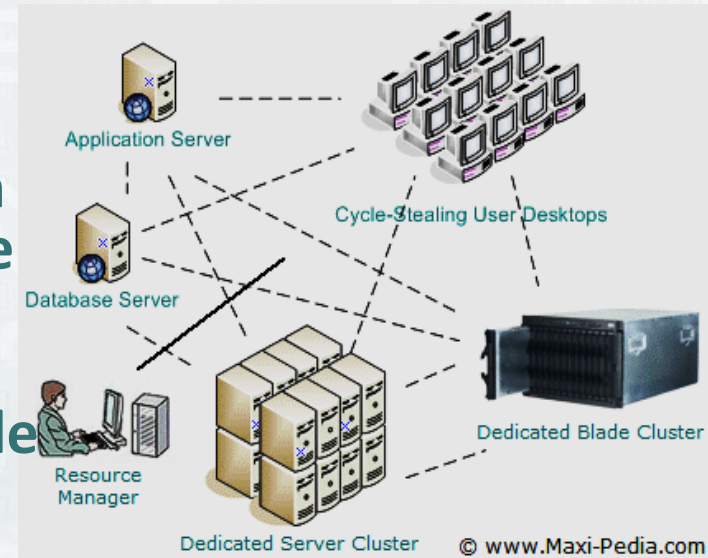
- Computación paralela (se estudia en AC)
 - Estudia los aspectos hardware y software relacionados con el *desarrollo y ejecución de aplicaciones* en un sistema de cómputo compuesto por **múltiples cores/procesadores/computadores** que es visto externamente como una **unidad autónoma** (multicores, multiprocesadores, multicomputadores, cluster)
- Computación distribuida
 - Estudia los aspectos hardware y software relacionados con el *desarrollo y ejecución de aplicaciones* en un **sistema distribuido**; es decir, en una **colección de recursos autónomos** (PC, servidores -de datos, software, ...-, supercomputadores ..., bases de datos distribuidas, sistemas de ficheros distribuidos, llamadas a procedimientos remotos) situados en **distintas localizaciones físicas** (peer-to-peer o P2P, cliente –servidor, n-tier, ...)



Computación distribuida a gran escala: Computación grid

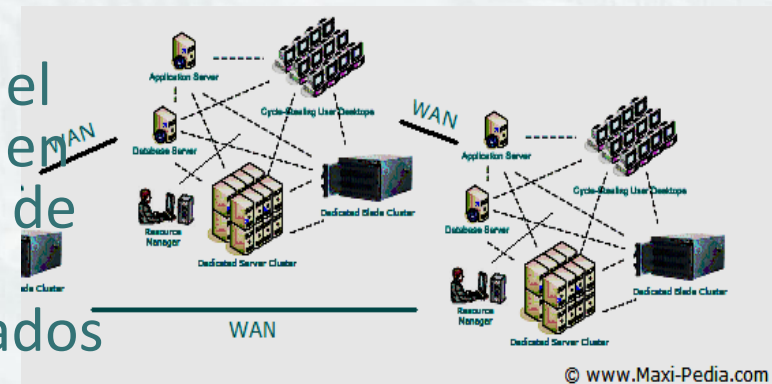
➤ Computación distribuida baja escala

- Estudia los aspectos relacionados con el *desarrollo y ejecución de aplicaciones* en una colección de recursos autónomos **de un dominio administrativo** situados en **distintas localizaciones físicas** conectados a través de **infraestructura de red local**



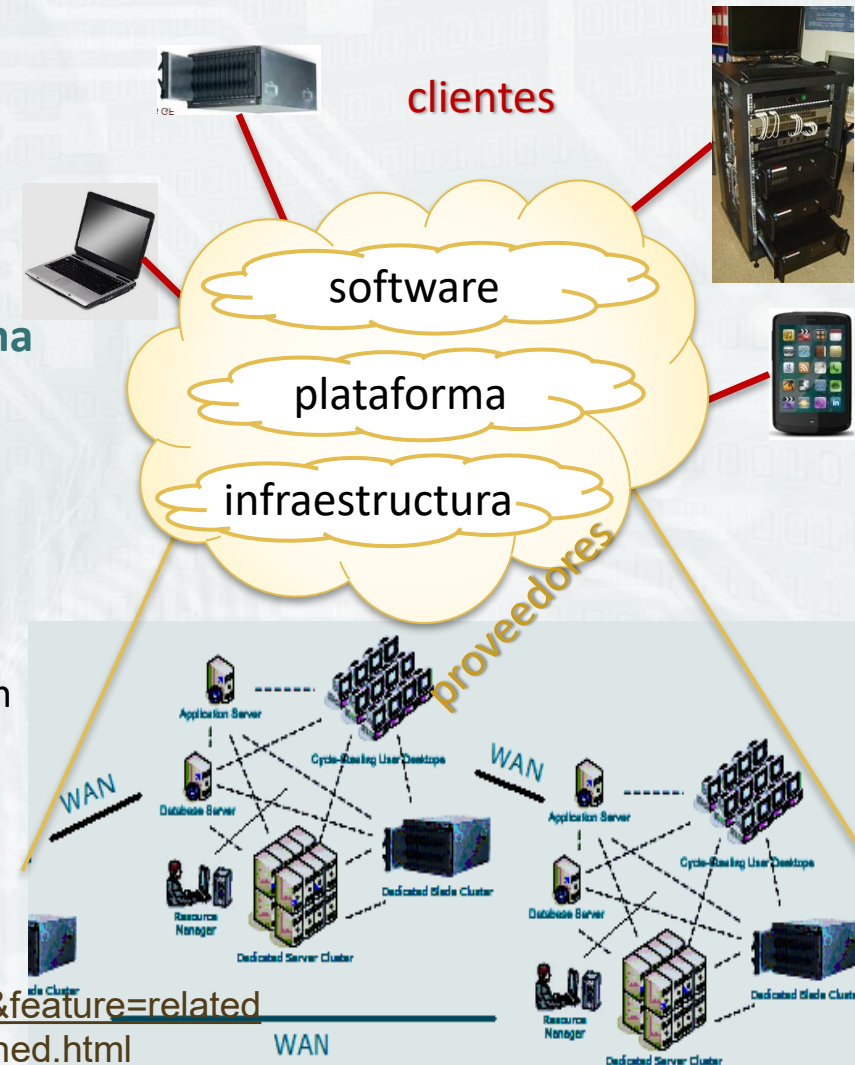
➤ Computación grid

- Estudia los aspectos relacionados con el *desarrollo y ejecución de aplicaciones* en una colección de recursos autónomos **de múltiples dominios administrativos** geográficamente distribuidos conectados con **infraestructura de telecomunicaciones**



Computación distribuida a gran escala: Computación nube o *cloud*

- Computación *cloud*
 - Comprende los aspectos relacionados con el desarrollo y ejecución de aplicaciones en un **sistema cloud**
- Sistema *cloud*
 - Ofrece **servicios de infraestructura, plataforma y/o software**, por los que se paga cuando se necesitan (**pay-per-use**) y a los que se accede típicamente a través de una **interfaz (web) de auto-servicio**
 - Consta de **recursos virtuales** que
 - son una **abstracción** de los recursos físicos
 - parecen **ilimitados en número y capacidad** y son reclutados/liberados de forma **inmediata** sin interacción con el proveedor
 - soportan el acceso de múltiples clientes (**multi-tenant**)
 - están conectados con métodos **estándar independientes** de la **plataforma de acceso**.



Para ampliar: <http://www.youtube.com/watch?v=SgujalzkwrE&feature=related>

<http://soa-eda.blogspot.com/2010/05/cloud-computing-explained.html>

<https://cs.unibo.it/babaoglu/courses/wsc/1-Intro-Economics.pdf>

Contenidos

- Lección 1. Clasificación del paralelismo implícito en una aplicación
- **Lección 2. Clasificación de arquitecturas paralelas**
 - Computación paralela y computación distribuida
 - **Clasificaciones de arquitecturas y sistemas paralelos**
 - Nota histórica
- Lección 3. Evaluación de prestaciones

Criterios de clasificación de computadores

→ Comercial

- Segmento del mercado
 - embebidos, servidores gama baja ...
- Educación, investigación (también usados por fabricantes y vendedores)
 - Flujos de instrucciones y flujos de datos: clasificación de Flynn (1972)
 - Sistema de memoria
 - Flujos de instrucciones (propuesta de clasificación de arquitecturas con múltiples flujos de instrucciones)
 - Nivel del paralelismo aprovechado (propuesta de clasif.)

Segmento del mercado (datos ventas 2021)



Clasificación de Computadores según segmento

Externo (*desktop, laptop, server, cluster ...*)
- R.EAC, IC.SCAP

Para todo tipo de aplicaciones:

- Oficina, entretenimiento, ...
- Procesamiento de transacciones o OLTP, sistemas de soporte de decisiones o DSS, e-commerce, ...
- Científicas (medicina, biología, predicción del tiempo, etc.) y animación (películas animadas, efectos especiales, etc.), ...

Empotrado (oculto) - IC.SCAE

Aplicaciones de propósito específico

- Videojuegos, teléfonos, coches, electrodomésticos, ...

Restricciones típicas:

- Consumo de potencia, precio, tamaño reducidos
- Tiempo real

Criterios de clasificación de computadores

➤ Comercial

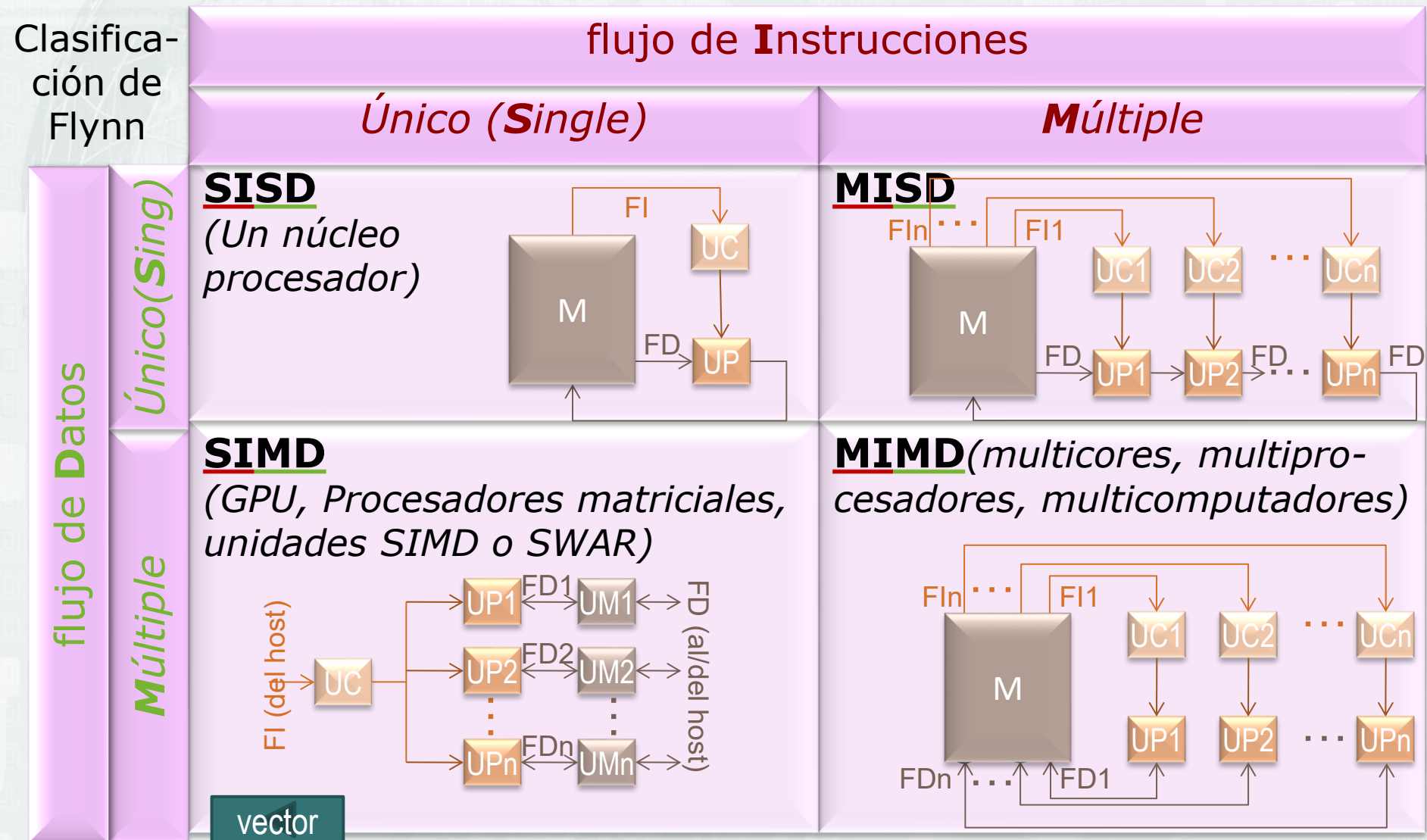
➤ Segmento del mercado

- embebidos, servidores gama baja ...

➔ Educación, investigación (también usados por fabricantes y vendedores)

- Flujos de instrucciones y flujos de datos (clasificación de Flynn 1972)
- Sistema de memoria
- flujos de instrucciones (propuesta de clasificación de arquitecturas con múltiples flujos de instrucciones)
- Nivel del paralelismo aprovechado (propuesta de clasif.)

Clasificación de Flynn de arquitecturas, 1972 (Flujo instr./flujo de datos)



Criterios de clasificación de computadores

➤ Comercial

➤ Segmento del mercado

- embebidos, servidores gama baja ...

➤ Educación, investigación: también usados por fabricantes y vendedores

➤ Flujos de instrucciones y flujos de datos (clasificación de Flynn 1972)

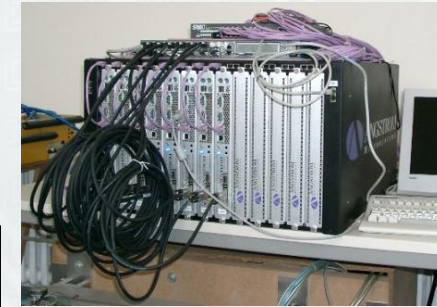
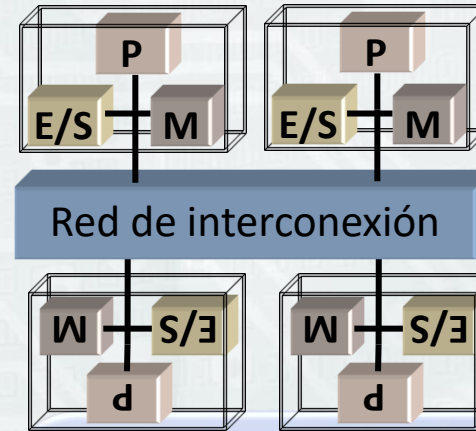
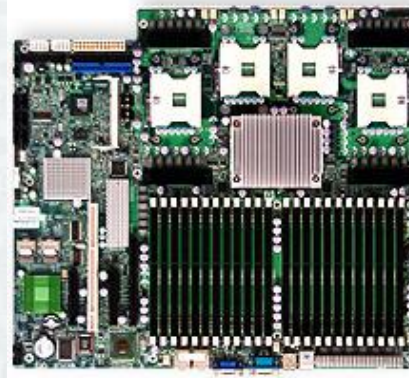
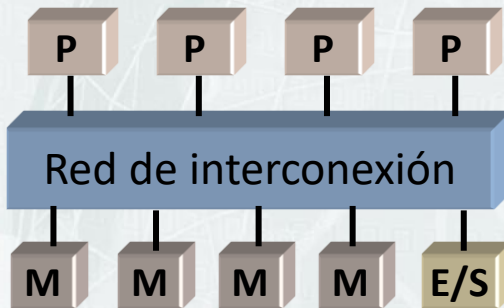
➡ Sistema de memoria

➤ Flujos de instrucciones (propuesta de clasificación de arquitecturas con múltiples flujos de instrucciones)

➤ Nivel del paralelismo aprovechado (propuesta de clasif.)

Clasificación de Computadores Paralelos

MIMD según el sistema de memoria



Multiprocesadores (62)

Todos los procesadores comparten el **mismo espacio de direcciones**

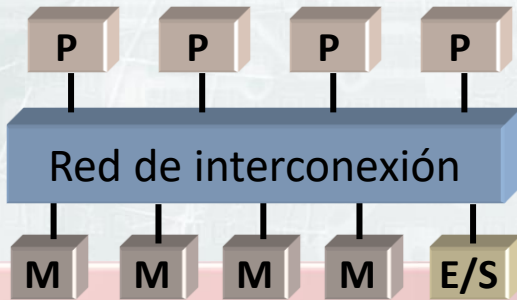
El programador NO necesita conocer dónde están almacenados los datos

Multicomputadores (85)

Cada procesador tiene su **espacio de direcciones propio**

El programador necesita conocer dónde están almacenados los datos

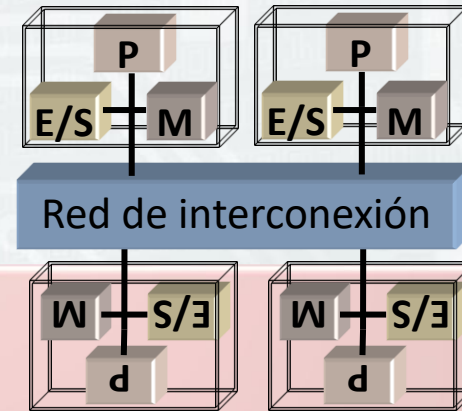
Comparativa SMP (*Symmetric Multi-Processor*) y multicomputadores I



Multiprocesador con memoria centralizada (62,SMP)

Mayor latencia - Poco escalable

Comunicación implícita mediante variables compartidas. Datos no duplicados en memoria principal

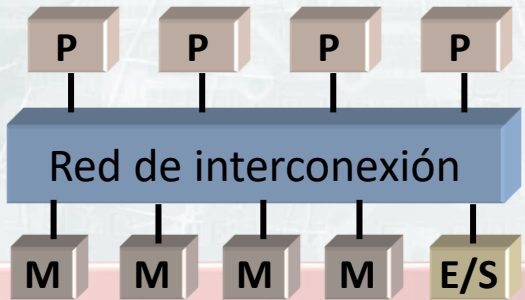


Multicomputador (85)

Menor latencia – Más escalable

Comunicación explícita mediante software para paso de mensajes (*send/receive*). Datos duplicados en memoria principal, copia datos

Comparativa SMP (*Symmetric Multi-Processor*) y multicomputadores II

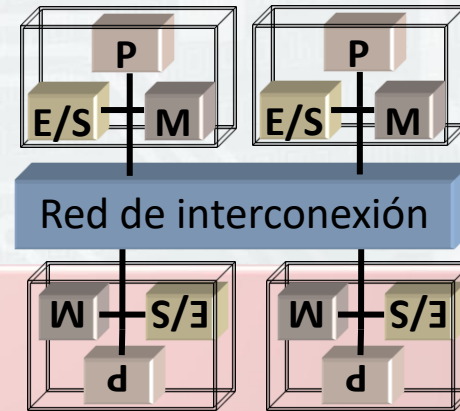


Multiprocesador con memoria centralizada (62,SMP)

Necesita implementar primitivas de sincronización

Distribución código y datos entre procesadores: no necesaria

Programación, generalmente, más sencilla



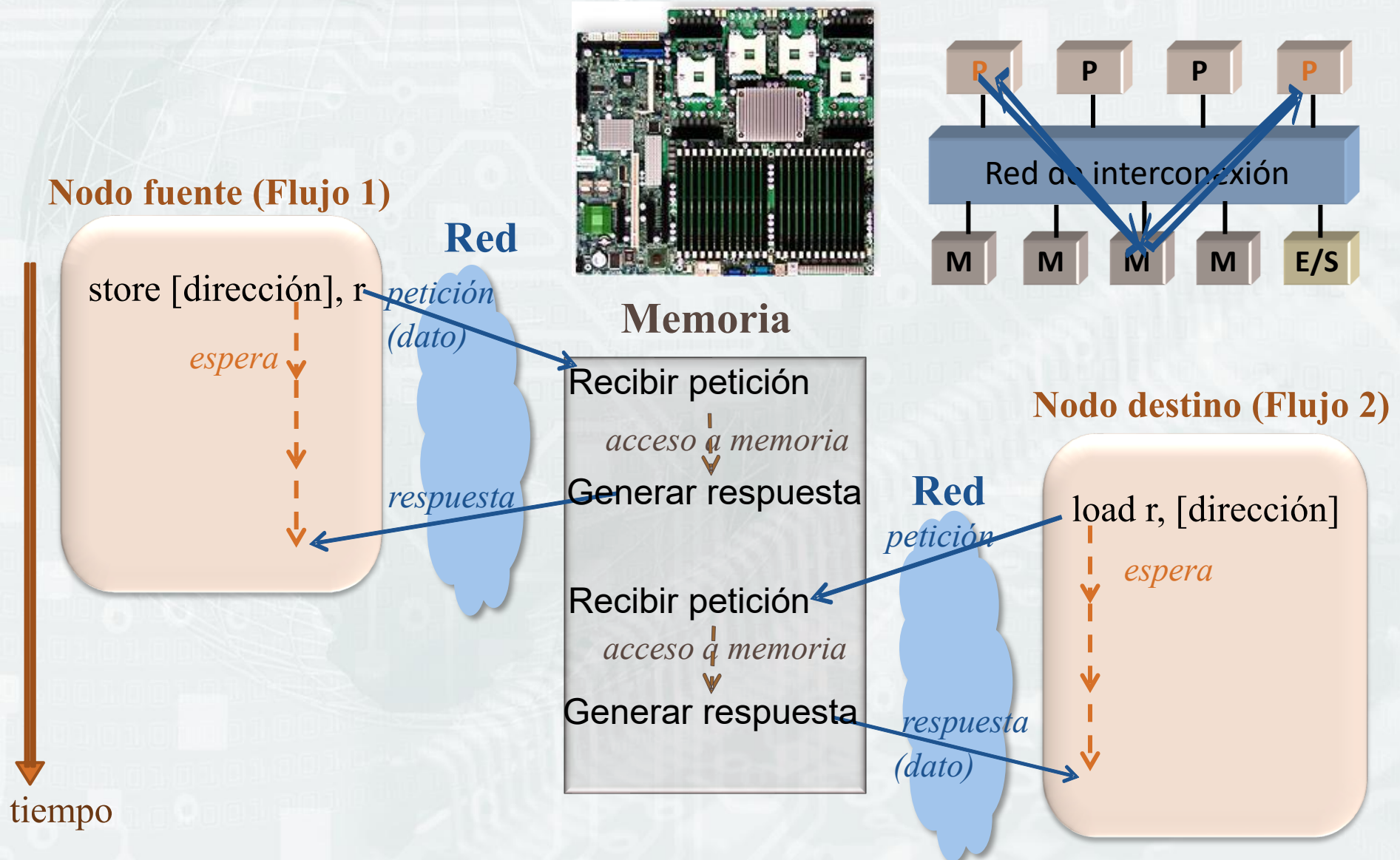
Multicomputador (85)

Sincronización mediante software de comunicación

Distribución código y datos entre procesadores: necesaria=> herramientas program. más sofisticadas

Programación generalmente más difícil

Comunicación uno-a-uno en un multiprocesador



Comunicación uno-a-uno en un multiprocesador

Secuencial	Paralelo	
...	<u>F1</u>	<u>F2</u>
A =valor;	...	...
...	A =valor;	copia= A ;
copia= A ;	...	...
...		

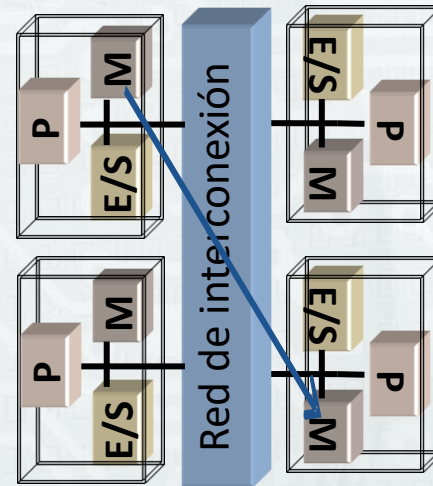
F1 es el flujo de instrucciones productor del dato (envía el dato)
F2 es el flujo de instr. consumidor del dato (recibe el dato)

- Se debe garantizar que el flujo de instrucciones consumidor del dato lea la variable compartida (A) cuando el productor haya escrito en la variable el dato

Paralelo multiproc. (K=0)	
<u>F1</u>	<u>F2</u>
...	...
A =valor;	while (K ==0) { };
K =1;	copia= A ;
...	...

NOTA: La programación paralela de multiprocesadores se estudia en AC

Comunicación uno-a-uno en multicomputador (*receive* bloqueante)



Nodo fuente (Flujo 1)

`send(dest, dat, size)`

continúa ejecución

copia datos a buffer

Datos

Red

Nodo destino (Flujo 2)

`receive(fuen, dat, size)`

espera

datos a memoria usuario

continúa ejecución

tiempo

Comunicación uno-a-uno en un multicomputador

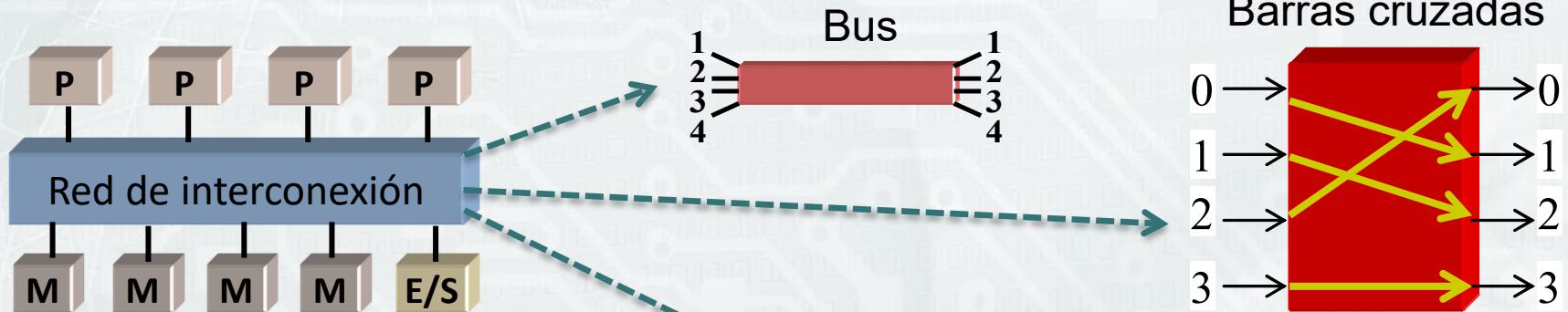
Secuencial	Paralelo	
...	<u>F1</u>	<u>F2</u>
A =valor;	...	...
...	A =valor;	copia= A ;
copia= A ;	...	...
...		

F1 es el flujo de instrucciones productor del dato (*envía* el dato)
F2 es el flujo de instr. consumidor del dato (*recibe* el dato)

Paralelo multicomputador (size = 4 byte)	
<u>F1</u>	<u>F2</u>
...	...
send(F2, valor, 4);	receive(F1,copia,4);
...	...

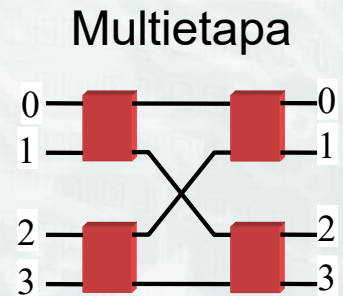
NOTA: La programación paralela de multicomputadores se estudia en la asignatura: **Arquitecturas y Computación de Altas Prestaciones** (IC.SCAP.ACAP – Especialidad (IC), Materia (SCAP), Asignatura (ACAP))

Incremento de escalabilidad en multiprocesadores y red de interconexión

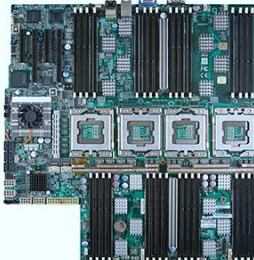
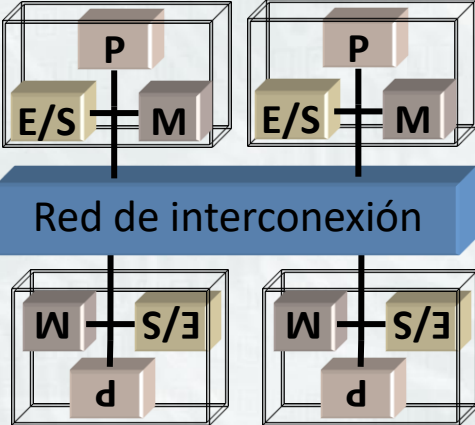
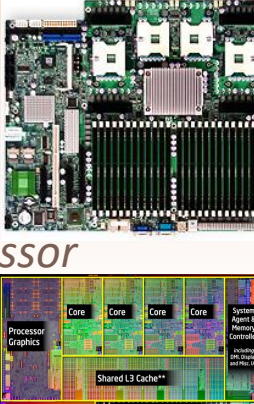
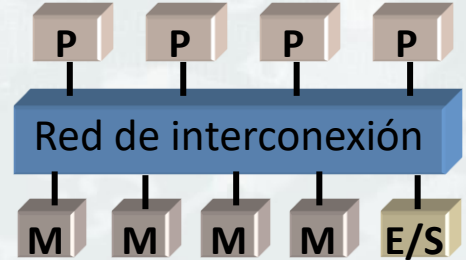


➤ Incremento escalabilidad multiprocesadores:

- Aumentar cache del procesador
- Usar redes de menor latencia y mayor ancho de banda que un bus (jerarquía de buses, barras cruzadas, multietapa)
- Distribuir físicamente los módulos de memoria entre los procesadores (pero se sigue compartiendo espacio de direcciones)



Clasificación completa de computadores según el sistema de memoria

Multi-computadores Memoria no compartida	NORMA <i>No Remote Memory Access</i>	<i>ej. cluster, red de computadores</i> 	Memoria físicamente distribuida	+ +
Multi-procesadores Memoria compartida Un único espacio de direcciones	NUMA <i>Non-Uniform Memory Access</i>	NUMA 		Nivel de empaquetamiento y conexión Escalabilidad
		CC-NUMA		
		COMA		
	UMA <i>Uniform Memory Access</i>	SMP <i>Symmetric MultiProcessor</i> 	Memoria físicamente centralizada 	

Criterios de clasificación de computadores

- Comercial
 - Segmento del mercado
 - embebidos, servidores gama baja ...
- Educación, investigación: también usados por fabricantes y vendedores
 - Flujos de instrucciones y flujos de datos (clasificación de Flynn 1972)
 - Sistema de memoria
 - Flujos de instrucciones (propuesta de clasificación de arquitecturas con múltiples flujos de instrucciones)
- ➔ Nivel del paralelismo aprovechado (propuesta de clasif.)

Arquitecturas con DLP, ILP y TLP (thread=flujo de control o de instrucciones)

Arq. con **DLP** (*Data Level Parallelism*)

Ejecutan las *operaciones* de una instrucción concurr. o en paralelo

Unidades funcionales vectoriales o SIMD (SWAR)

Arq. con **ILP** (*Instruction Level Parallelism*)

Temas(1, 4), BP4

Ejecutan múltiples *instrucciones* concurr. o en paralelo

Cores escalares segmentados, superescalares o VLIW/EPIC

Arq. con **TLP** (*Thread Level Parallelism*) explícito y una instancia de SO

Temas(1-3), BP(0-3)

Ejecutan múltiples *flujos de instrucciones* concurr. o en paralelo

Cores que modifican la archit. escalar segmentada, superescalar o VLIW/EPIC para ejecutar threads concurr. o en paralelo

Multi-procesadores: ejecutan threads en paralelo en un computador con múltiples cores (incluye **multicores**)

Arq. con **TLP** explícito y **múltiples** instancias SO

IC.SCAP

Ejec. múltiples *flujos de instr.* en paralelo

Multi-computadores : ejecutan threads en paralelo en un sistema con múltiples computadores

Contenidos

- Lección 1. Clasificación del paralelismo implícito en una aplicación
- **Lección 2. Clasificación de arquitecturas paralelas**
 - Computación paralela y computación distribuida
 - Clasificaciones de arquitecturas y sistemas paralelos
 - **Nota histórica**
- Lección 3. Evaluación de prestaciones

Nota histórica. DLP y ILP

➤ DLP (*Data Level Parallelism*)

➤ Unidades funcionales (o de ejecución) SIMD (o multimedia)

- **1989** (*Intel i860*). **1991** (motorola M88110). **1993** (repertorio MAX en HP PA : PA7100LC). **1995** (repertorio VIS en Sun Sparc: Ultra I). **1997** (repertorio MMX en Intel x86: Pentium MMX). **1999** (repertorio SSE en Intel x86: Pentium III; repertorio AltiVec en IBM Power: PowerPC 8000)

➤ ILP (*Instruction Level Parallelism*) :

TEMA 4

➤ Procesadores/cores segmentados

- **1961** (*IBM 7030*). **1982** (chip Intel i286, Motorola 68020). **1986** (chip MIPS R2000). **1987** (chip AMD Am29000). **1988** (chip Sun Sparc)...

➤ Procesadores con múltiples unidades funcionales

- **1967** (IBM 360/91) ...

➤ Procesadores/cores superescalares

- **1989** (chip Intel 960CA (3)). **1990**: (chip IBM Power1 (4)). **1992**: (chips DEC α 21064 (2/4), HP PA 7100 (2/2), Sun SuperSparc (3/5)). **1993**: (Pentium(2/3)) **1995**: (Pentium Pro (3/7)) ...

➤ Procesadores/cores VLIW

- **1990** (*chip Intel i860 (2)*). **1997** (*chip DSP TMS320C6x (8)*). **2001** (chip Intel Itanium)...

NOTA: Destacado en *cursiva* las primeras implementaciones y en *color* más claro los chip de propósito específico

Nota histórica. TLP (*Thread Level Parallelism*)

TEMA 3

- TLP explícito con una instancia de SO:
 - Multithread grano fino (FGMT)
 - **1975** (*Denelcor HEP*). **2005** (chip Sun UltraSPARC T1) ...
 - Multithread grano grueso (CGMT)
 - **1990** (*MIT Alewife*). **2000** (chip IBM PowerPC RS64 IV (2)). **2006** (chip Intel Itanium Montecito (2)) ...
 - Multithread simultánea (SMT)
 - **2002** (chip Intel Pentium 4/Xeon Hyper-Threading). **2004** (chip IBM Power5) ...
 - Multiprocesadores en un chip (CMP) o multicores
 - **2001** (chip IBM Power4). **2004** (chip Sun UltraSPARC IV). **2006** (chip Intel Core Duo). **2008** (chip Intel Celeron Dual-core) ...
 - Multiprocesadores
 - **1962** (*Burroughs D825 - red barras cruzadas*). **1966** (*UNIVAC 1108 - red bus*). **1985** (*IBM RP3 - red multietapa*). **1996** (*SGI Origin 2000 -CC-NUMA, red estática+multietapa*). **2006** (SGI Altix 4000) ...
- TLP explícito con múltiples instancias del SO (multicomputadores) IC.SCAP
 - **1985** (*Intel iPSC1 - i286+red estática con Ethernet*)... cualquier cluster

NOTA: Destacado en *cursiva* las primeras implementaciones y en **color** más claro los chip de propósito específico